

Reviewer Pack — Minimal Number of Affine Plane Points and Frege Proof Depth ...

Entry 216401784e29 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 22:17:22 UTC

Minimal Number of Affine Plane Points and Frege Proof Depth Inequality

Entry ID: 216401784e29 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 22:17:22 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

For every Boolean formula φ with m clauses and n variables, the depth $d(\varphi)$ of a Frege proof for φ is linearly related to the minimal number of points in an affine plane that can be used to represent φ , such that $d(\varphi) = \Theta(\log(m + n))$

Rationale (proposer's reasoning):

Affine geometry provides a rich framework for geometric representations of combinatorial structures. Points in an affine plane have been used as a model for representing Boolean functions. The conjecture aims to link the combinatorial complexity of Frege proofs with the geometric properties of these points, potentially revealing new insights into proof complexity.

Taxonomy category: affine_geometry_frege_lowerbound (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3571b4f4d58b390f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal number of affine plane points and Frege proof depth exceeds 0.7 across all tested Boolean formulas, and no seed produces a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine geometry" AND "Frege proof complexity" - "Boolean satisfiability" AND "affine plane points" - "Frege proof depth inequality" AND "minimal number of points in affine plane"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(m, n):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(m):
            clause = random.sample(variables + [f'~{v}' for v in variables], 2)
            clauses.append(clause)
        return clauses

    def count_affine_plane_points(clauses, variables):
        points = set()
        for clause in clauses:
            for var in clause:
                if var.startswith('~'):
                    points.add(var[1:])
                else:
                    points.add(var)
        return len(points)

    def frege_proof_depth(clauses):
        n = len(clauses)
        d = 0
        while n > 1:
            n = math.ceil(n / 2)
            d += 1
        return d

    m_values = [5, 10, 15, 20, 30, 40]
```

```

results = []

for m in m_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        n = random.randint(2, min(10, m)) # Ensure n_min >= 5
        formula = generate_boolean_formula(m, n)
        points = count_affine_plane_points(formula, variables)
        depth = frege_proof_depth(formula)
        results.append({
            "m": m,
            "n": n,
            "points": points,
            "depth": depth
        })

if not results:
    return {
        "metric_name": "Frege Proof Depth vs. Affine Plane Points",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

points = [r["points"] for r in results]
depths = [r["depth"] for r in results]

def correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    var_x = sum((x[i] - mean_x) ** 2 for i in range(n)) / n
    var_y = sum((y[i] - mean_y) ** 2 for i in range(n)) / n
    return cov / (math.sqrt(var_x) * math.sqrt(var_y))

corr_coeff = correlation(points, depths)

return {
    "metric_name": "Frege Proof Depth vs. Affine Plane Points",
    "metric_value": corr_coeff,
    "instances_tested": len(results),
    "n_max": max(r["n"] for r in results),
    "conjecture_holds": 0.5 < corr_coeff < 0.7,
    "counterexample": "" if 0.5 < corr_coeff < 0.7 else f"Correlation coefficient: {corr_coeff}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

if not all(r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient out of range\" first_fa
else:
    mean_corr_coeff = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if 0.5 < r["metric_value"] < 0.7) / len(results)
    print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std=0 support_fraction={support_fraction}

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b1396b9.py", line 102, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b1396b9.py", line 54, in run_trial
    points = count_affine_plane_points(formula, variables)
                                           ^^^^^^^^^
NameError: name 'variables' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to compute the correlation coefficients required to evaluate the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, re-run the test with a valid dataset to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13842
2	propose	ollama_remote	glm4:latest	0	0	11721
3	propose	ollama_remote	glm4:latest	0	0	14687
4	propose	ollama_remote	glm4:latest	0	0	12672
5	preregistration	ollama_remote	glm4:latest	0	0	9016
6	novelty	ollama_remote	glm4:latest	0	0	8169

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	novelty	ollama_remote	glm4:latest	0	0	8853
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17018
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11833
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11991
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11184
12	judge	ollama_remote	glm4:latest	0	0	17482

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148469 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in *research/audit/216401784e29.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/216401784e29.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/216401784e29.tar.gz* (if generated)